



# Cargo Sherlock: An SMT-Based Checker for Software Trust Costs

Muhammad Hassnain  
University of California, Davis  
Davis, CA, USA  
mhassnain@ucdavis.edu

Ethan Ng  
University of California, Davis  
Davis, CA, USA  
eyng@ucdavis.edu

Anirudh Basu  
University of California, Davis  
Davis, CA, USA  
anibas@ucdavis.edu

Caleb Stanford  
University of California, Davis  
Davis, CA, USA  
cdstanford@ucdavis.edu

## Abstract

Supply chain attacks threaten open-source software ecosystems. This paper proposes a framework for quantifying trust in third-party software dependencies that is *formally checkable* – formalized in satisfiability modulo theories (SMT) – while at the same time incorporating *human factors*, like the number of downloads, authors, and other metadata that are commonly used to identify trustworthy software in practice. We use data from software analysis tools and metadata to build a first-order relational model of software dependencies; to obtain an overall “trust cost” combining these factors, we propose a formalization based on the *minimum trust problem*, which asks for the minimum cost of a set of assumptions that can be used to conclude that the code is safe. We implement these ideas in Cargo Sherlock, targeted for Rust libraries (crates), incorporating a list of candidate assumptions motivated by quantifiable trust metrics identified in prior work. Our evaluation shows that Cargo Sherlock can identify synthetically generated supply chain attacks and known incidents involving typosquatted and poorly AI-maintained crates. Its performance scales to Rust crates with many dependencies.

## CCS Concepts

• **Software and its engineering** → **Formal methods**; • **Security and privacy** → **Software and application security**; **Human and societal aspects of security and privacy**.

## Keywords

Formal Methods, Trust in Software, Human Factors in Security, Software Supply Chain

## ACM Reference Format:

Muhammad Hassnain, Anirudh Basu, Ethan Ng, and Caleb Stanford. 2026. Cargo Sherlock: An SMT-Based Checker for Software Trust Costs. In *IEEE/ACM 14th International Conference on Formal Methods in Software Engineering (FormalISE '26)*, April 12–13, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3793656.3793687>

## 1 Introduction

As software ecosystems grow in size and complexity, automated techniques are needed to prevent supply chain attacks [40, 45]. Recent and notable examples of such attacks include the SolarWinds attack [40]; the XZ Utils backdoor [14, 72]; and for Rust, which is the focus of our implementation, the malicious libraries *rustdecimal* [51] and *faster\_log* [47]. These attacks occur at the *package granularity*, where untrustworthy libraries or software dependencies are added to trustworthy projects, or safe libraries are hijacked by malicious developers. Many existing techniques can be used to help identify such attacks. Some of these are *formally checkable* in the sense that they are based on a well-defined formal model of software: e.g., static analysis tools, fuzzers, type systems, and verification tools. For such tools, analysis results correspond to concrete syntactic or semantic source code properties or program traces, and can therefore be understood in an appropriate formal system [42]. However, these tools are often limited to specific semantic or syntactic properties of programs.

On the other hand, industry developers and security engineers typically rely on a rather different set of criteria in practice: they use a combination of *trust* in software and *auditing* to ensure that third-party dependencies are safe from software supply chain attacks [7, 12, 21, 43, 71]. For example, developers at a large company may be likely to trust software that was developed by that company or one of its industry partners; at a smaller scale, an independent developer may be more inclined to trust software that is widely accepted by the community (e.g., has a large number of downloads and stars on GitHub). Trust and auditing are the foundation of techniques like software bill of materials (SBOM) [23, 44, 66, 67], which require human-curated or manually generated lists of software components and their purposes. However, trust and auditing (as currently practiced) often come with no formal guarantees; there is no formal model over which we can understand the meaning of audits and trust relationships, and how they compose.

**Research goals.** Is it possible to quantify trust in a way that is formally checkable (in some formal system) yet also accounts for human factors that many developers rely on in practice? Our specific focus is to quantify trust in propositional and first-order logic; roughly speaking, the more assumptions that must be taken for a software dependency “on faith”, the greater the *trust cost* that we should need to pay in order to believe it functions correctly. It should be the case that higher trust costs correspond to more risky code. We identify the following specific goals for such a trust cost:



This work is licensed under a Creative Commons Attribution 4.0 International License. *FormalISE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2478-7/2026/04  
<https://doi.org/10.1145/3793656.3793687>

- Trust should be *specific*: a trust cost should commit the analysis to a specific assumption or set of assumptions about the software (e.g., the number of downloads, trusted audits, or the results of specific analysis tools).
- Trust should be *auditable*: it should be possible for a human to read and understand the assumptions made by a trust analysis.
- Trust should be *configurable*: the user should be able to add and revise assumptions, e.g., by adding additional tool outputs or human factors as parameters.
- Finally, trust should be *composable*: we should be able to compare and combine trust costs corresponding to different portions of the package ecosystem.

We note that traditional software vulnerability analysis and other machine learning (ML) approaches [15, 32] are a non-solution to the above. Such ML techniques would meet our criteria for configurability and composability; but they would fail the first and second criteria, auditability and specific assumptions that can later be vetted by human developers and human users.

**Cargo Sherlock: an SMT-based approach.** To address these goals, we propose Cargo Sherlock, a design and implementation for quantifying supply chain trust based on a formalization in satisfiability modulo theories (SMT). Cargo Sherlock can be used directly by end users to quantify trust when selecting dependencies to add to a project, and its features are motivated by this intended use case; see Figure 1 and Section 2.

We begin with an SMT-based formalization of trust that we call the *minimum trust problem* (Section 3). This particular formalization assumes that we have a list of assumptions that the user might make about the code (such as: “if the code has a large number of downloads, then it is safe” or “if the code is audited by a trusted organization, then it is safe”); each assumption has an associated cost, which can be thought of as the cost inherent in making that assumption. These assumptions are motivated by prior empirical research; importantly, they are not necessarily true facts (if they were true unconditionally then trust would not be required!), but rather, they are real thought processes that users might potentially employ to conclude that dependencies are safe, and are configurable by users. The minimum trust problem then is to determine the *minimum cost* of a set of assumptions, encoded in propositional logic, that logically entail that the code is safe. We argue that this model can meet the goals we outlined above; for example, it produces trust costs that are specific to a particular set of assumptions, can be configured to meet specific users’ needs, and are auditable by human developers. We consider two algorithms for solving the minimum trust problem, based on a reduction to SMT. We also show that the problem is NP-hard in the worst case. This formalization is similar in principle to existing work from the logic programming and databases communities, for example, the semiring framework for database provenance [19], assigning weights to logical conclusions in Datalog [5], and solvers for weighted Max-SAT [24, 37].

For the design and implementation (Section 4), Cargo Sherlock gathers software supply chain metadata and the results of automated program analysis tools to build a *first-order relational structure* of the software supply chain – that is, of all the dependencies of a given package. It then instantiates the minimum trust problem

above on a set of possible assumptions on the variables involved in this first-order structure. The implementation also supports “parameterized assumptions”, where the cost depends in a continuous way on a particular piece of metadata, and “negative assumptions”, which can be used to incorporate information about software components that are *distrusted*, such as known vulnerabilities or bugs found by the program analysis tools.

Our evaluation (Section 5) asks four questions: Can Cargo Sherlock be used to prevent synthetically generated supply chain attacks? Would Cargo Sherlock identify risky Rust dependencies (crates) that have been involved in known supply chain incidents? How does Cargo Sherlock fare on known vulnerable crates? Lastly, does the performance of Cargo Sherlock scale to Rust crates with many dependencies? Among our results, we find that synthetically generated typosquatting attacks generally result in much higher trust costs. We also find that of the two algorithms we propose, our more optimized algorithm provides timely results for crates with a large number of dependencies.

One way to put these results into context is to think about a particular *user model* of how we expect users to interact with our tool. For example, due to the higher trust costs, if using our tool in the prescribed model, a user would catch any of the typosquatting attacks considered in our evaluation by identifying and omitting the risky dependency prior to its inclusion.

#### Contributions in summary:

- A formal model of the supply chain trust problem and two algorithms for its solution using the SMT theory of linear arithmetic (Section 3);
- A design and architecture for a usable tool based on these ideas, including concrete input data about Rust code and a collection of built-in candidate assumptions motivated by human factors and available program analyses (Section 4);
- An evaluation of the efficacy of our tool for identifying synthetic and real-world supply chain attacks, and its scalability on Rust libraries with many dependencies (Section 5).

We conclude the paper with related work (Section 6). Cargo Sherlock is open source (MIT license) and available via a virtual machine image on Figshare<sup>1</sup> and as a repository on GitHub.<sup>2</sup>

## 2 Motivating Overview

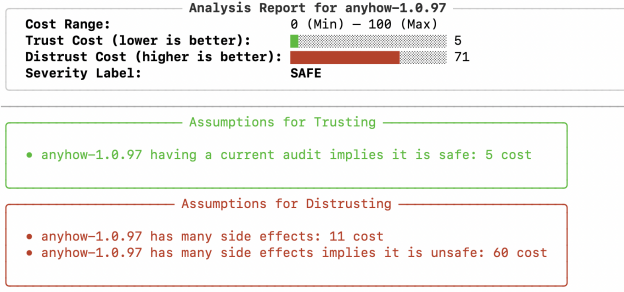
This section presents a motivating overview of Cargo Sherlock from a user-facing perspective; the reader who is more interested in formal details may skip to Section 3. We consider a hypothetical example of Ferris, a Rust developer who wishes to install a Rust crate called `aho_corasick` (implementing the Aho-Corasick algorithm), and explore how Cargo Sherlock’s approach might help Ferris attain the goals laid out in the introduction.

### 2.1 Automating trust costs

After finding the crate `aho_corasick` from a Google search, Ferris faces a choice: do they trust the crate blindly, or do they first try to determine if the crate is safe to install on their system? If Ferris is a typical developer, they might employ a common middle ground by

<sup>1</sup><https://doi.org/10.6084/m9.figshare.31094197>

<sup>2</sup><https://github.com/davispl/cargo-sherlock>



**Figure 1: Cargo Sherlock example output on the anyhow crate. (Note that lower trust cost, higher distrust cost is better.)**

relying on various heuristics to argue that the crate is likely safe: e.g., that the crate has many downloads, or that it is developed by someone who is trustworthy. At its core, our tool supports Ferris by attempting to *automate* this reasoning. The premises of Ferris’ arguments are based on *assumptions* about the supply chain ecosystem (e.g., “the more downloads a crate has, the safer it is”); Cargo Sherlock includes a default list of such assumptions motivated by prior research (see Section 4.3). Ferris may find some assumptions more reasonable than others; we mark each assumption with a *cost* to reflect its reasonableness. If Ferris runs Cargo Sherlock on `aho_corasick`, it will present them with `aho_corasick`’s *trust cost*: the total cost of the argument whose premises have the lowest total cost to assume (informally, the *strongest* argument). After using Cargo Sherlock, Ferris determines that `aho_corasick` is marked as trusted, and that it was marked SAFE due to having a high number of downloads. See Figure 1 for an example output; the trust cost (lower is better) is shown at the top in green.

## 2.2 Trust should be specific

Now suppose that Ferris becomes concerned about trusting crates with a high number of downloads. To address this concern, Ferris hires an expert auditor, Crabby, who combs through every line of source code of `aho_corasick`. Since Ferris trusts Crabby’s expertise, Ferris assigns a very low cost to the assumption that “if the crate is audited by Crabby, it is safe” – lower than any other available assumption. Next, suppose Crabby comes back after the audit and reports that the crate is completely safe to use. In this scenario, it would seem that the previous assumptions used to mark the crate trusted – such as its high number of downloads or the author of the crate – are no longer relevant to Ferris. Since Cargo Sherlock only reports the *minimum* trust cost for a crate with the associated assumptions, it does not present other (potentially unhelpful) assumptions. In other words, Ferris is only presented with a single *specific* reason to trust the crate. In Figure 1, the specific assumptions are shown under “Assumptions for Trusting” in green.

## 2.3 Trust should be auditable and configurable

Unfortunately, Ferris later finds out that Crabby is untrustworthy. Since this renders the existing audit suspect, Ferris revisits the previous assumptions, and increases the trust cost for “if the crate is audited by Crabby, it is safe”. In turn, Cargo Sherlock will update its result: the trust cost for `aho_corasick` will increase, and a new

minimum set of assumptions will be identified. This same mechanism can be used if Ferris wants to add new information to the tool (e.g., adding additional trusted authors and auditors).

## 2.4 Trust should be composable

During this process, Ferris discovers that `aho_corasick` depends on a second crate, `memchr`, that was previously overlooked. Since this could present an additional attack surface, Ferris believes that the trust cost for `aho_corasick` should be dependent on the cost of its dependencies in some cases. One way to model this is with assumptions such as the following one: “if a crate’s dependencies are all safe to use, and the crate does not have any code flagged by a static analysis tool, then the crate is safe.” Cargo Sherlock uses a similar assumption to allow arguments that compose trust costs on a crate’s dependencies to obtain a trust cost for the crate itself.

## 2.5 Incorporating known risks

Finally, Ferris discovers the RustSec Advisory Database [52], which lists vulnerabilities for different Rust crates. Ferris wants to check whether `aho_corasick` is present in the database. Unfortunately, nothing we have considered so far accounts for *negative* factors that would decrease trust in a crate; intuitively, this is because we have only incorporated assumptions that are about trustworthiness. We thus extend the tool with a *distrust* model to quantify known risks; we combine the results of the trust and distrust models to obtain a final label for the crate. In Figure 1, the distrust model is shown alongside the trust model with output in red.

## 3 Minimum Trust Problem

In this section, we present the formal model behind Cargo Sherlock, which aims to satisfy the requirements from Sections 1 and 2. We define the *minimum trust problem* formally: given a set of candidate assumptions and a conclusion, it asks to find the set of assumptions with minimum cost that entails the conclusion. We begin with an example, followed by two algorithms for the problem: first, a naive algorithm (Algorithm 1) and second, one based on Horn clause inference (Algorithm 2) that forms the basis for our trust solver implementation. To conclude the section, we show that the complexity of the minimum trust problem for Horn clauses is NP-complete, justifying such SMT-based approaches.

### 3.1 Problem definition

*Propositional logic basics.* A *propositional variable* represents a true or false statement that is indivisible into other true or false statements, such as “the library is safe.” We use lowercase letters to denote propositional variables. A *formula* in propositional logic is either a propositional variable, constant ( $\top$  for true,  $\perp$  for false), or built from propositional variables and logical connectives (“and”  $\wedge$ , “or”  $\vee$ , “not”  $\neg$ , “implies”  $\implies$ ). We use uppercase letters to denote formulas and uppercase Greek letters to denote sets of formulas. A *Horn clause* is a formula of the form  $(p_1 \wedge \dots \wedge p_n) \implies q$ ,  $\top \implies q$ , or  $(p_1 \wedge \dots \wedge p_n) \implies \perp$ . The antecedent of the conditional is called the *body* and the consequent is called the *head*. A Horn clause is a *definite clause* if it is in the form  $(p_1 \wedge \dots \wedge p_n) \implies q$ . A set of definite Horn clauses is *acyclic* if there exists a function  $f$  from propositional variables to natural numbers such that for every

| No. | Assumption                                                             | Encoding ( $A_i$ )        | Cost |
|-----|------------------------------------------------------------------------|---------------------------|------|
| 1   | aho_corasick is safe.                                                  | $c$                       | 100  |
| 2   | memchr is safe.                                                        | $m$                       | 100  |
| 3   | aho_corasick has many downloads.                                       | $d$                       | 30   |
| 4   | If aho_corasick has many downloads, then it is safe.                   | $d \implies c$            | 20   |
| 5   | Alice is trustworthy.                                                  | $a$                       | 20   |
| 6   | Bob is trustworthy.                                                    | $b$                       | 5    |
| 7   | If Alice is trustworthy and memchr is safe, then aho_corasick is safe. | $(a \wedge m) \implies c$ | 10   |
| 8   | If Bob is trustworthy, then memchr is safe.                            | $b \implies m$            | 10   |

**Table 1: Example instance of the minimum trust problem.**

definite Horn clause  $(p_1 \wedge \dots \wedge p_n) \implies q, f(q) > f(p_i)$  for all  $i$ . A *truth assignment* is an assignment of true or false to propositional variables. A set of formulas  $\Gamma$  *entails* another formula  $A$  (written as  $\Gamma \models A$ ) if and only if there is no truth assignment that makes all formulas in  $\Gamma$  true and  $A$  false. Equivalently,  $\Gamma \models A$  if and only if  $\Gamma \cup \{\neg A\} \models \perp$ . A set of formulas  $\Gamma$  is *satisfiable* if there exists some truth assignment that makes all formulas in  $\Gamma$  true.

**Definition 1** (Minimum trust problem). Given as input (1) a set of propositional formulas  $\Gamma = \{A_1, \dots, A_n\}$  representing assumptions, (2) a propositional variable  $c$  representing a conclusion, and (3) a function  $C: \Gamma \rightarrow \mathbb{N}$  that assigns a cost to making each assumption, output a set of assumptions  $\Delta \subseteq \Gamma$  such that the sum of the costs of assumptions in  $\Delta$  is minimized, subject to the constraint  $\Delta \models c$ .

We provide a simple example of the minimum trust problem, continuing the example from Section 2. Suppose that aho\_corasick depends on memchr, where aho\_corasick was developed by Alice, and memchr was developed by Bob and has no dependencies. Table 1 gives a set of candidate assumptions  $A_1, \dots, A_8$  a user can make about the safety of the aho\_corasick library and their associated costs in this scenario. We are asked to find a subset of  $A_i$  with minimum cost which entails  $c$ . In this example, the minimum cost is 45, incurred from assumptions  $a$  (cost 20),  $b$  (cost 5),  $(a \wedge m) \implies c$  (cost 10), and  $b \implies m$  (cost 10). Note that the set of assumptions is not necessarily unique in general (as there may be a tie for the minimum cost), though it is unique in this example.

### 3.2 Naive algorithm

Algorithm 1 shows a naive solution for the minimum trust problem using an SMT solver by encoding the constraints as sentences over linear integer arithmetic and using quantifiers over propositional variables. To associate each assumption being made to a cost, we use auxiliary propositional variables  $a_1, \dots, a_n$  to track each assumption  $A_1, \dots, A_n$ . Each  $a_i$  is assigned to true if and only if  $A_i$  is included in the solution set of assumptions  $\Delta$ . In the running example, we would have  $a_1 \implies c, a_4 \implies (d \implies c)$ , etc. We then encode the cost of the assumptions in  $\Delta$  as a sum of if-then-else (*ite*) terms; in the running example, the cost of any particular solution is given by  $\text{cost}(a_1, \dots, a_8) = \text{ite}(a_1, 100, 0) + \dots + \text{ite}(a_8, 10, 0)$ . The constraint that the conclusion  $c$  follows from the assumptions being made,  $\Delta \models c$ , is encoded as a formula  $\text{valid}(a_1, \dots, a_n)$ . In the running example,  $\text{valid}(a_1, \dots, a_8) = \neg \exists c, m, d, a, b ((a_1 \implies$

#### Algorithm 1 Naive minimum trust algorithm.

```

1: library: a software dependency in a package repository (e.g., a Rust
   crate on crates.io)
2: get_all_assumptions(library): a function that returns a set of all
   possible assumptions for library
3: C: a function that assigns a cost to an assumption
4: S: an SMT solver over the theory of linear integer arithmetic
5: procedure JoinAssumptions(lib)
6:    $F \leftarrow \top$ 
7:    $\Gamma \leftarrow \text{get\_all\_assumptions}(\text{lib})$ 
8:   for all  $A_i$  in  $\Gamma$  do
9:      $F \leftarrow F \wedge (a_{i|\text{lib}} \implies A_i)$ 
10:  for all dep in lib.dependencies do
11:     $F \leftarrow F \wedge \text{JoinAssumptions}(\text{dep})$   $\triangleright$  JoinAssumptions is
       called recursively on each dependency
12:  output  $F$ 
13: procedure SolveMinimumTrust()
14:    $F(a_1, \dots, a_n) \leftarrow \text{JoinAssumptions}(\text{library})$ 
15:    $\{v_1, \dots, v_m\} \leftarrow \text{atoms}(F) \setminus \{a_1, \dots, a_n\}$   $\triangleright$  atoms returns the set
       of propositional variables in a formula
16:    $\text{valid}(a_1, \dots, a_n) \leftarrow \neg \exists c \exists v_1 \dots \exists v_m (F(a_1, \dots, a_n) \wedge \neg c)$   $\triangleright c$ 
       is the conclusion 'the library is safe'
17:    $\text{cost}(a_1, \dots, a_n) \leftarrow \sum_{i=1}^n \text{ite}(a_i, C(a_i), 0)$ 
18:    $\text{minimal} \leftarrow \forall a_1 \dots \forall a_n (\text{valid}(a_1, \dots, a_n) \implies$ 
        $\text{cost}(a_1^*, \dots, a_n^*) \leq \text{cost}(a_1, \dots, a_n))$ 
19:   S.solve( $\text{valid}(a_1^*, \dots, a_n^*) \wedge \text{minimal}$ )  $\triangleright (a_1^*, \dots, a_n^*)$  is the
       solution to the minimum trust problem
20:  output S.model[ $a_1^*, \dots, a_n^*$ ], S.model[ $\text{cost}(a_1^*, \dots, a_n^*)$ ]

```

$c) \wedge \dots \wedge (a_8 \implies (b \implies m)) \wedge \neg c$ ). Finally, we encode using quantifiers the fact that  $\text{cost}(a_1, \dots, a_n)$  is minimized subject to the constraint  $\text{valid}(a_1, \dots, a_n)$ . In the running example, the SMT solver would produce a model that assigns  $a_5, a_6, a_7, a_8$  to true and  $a_1, a_2, a_3, a_4$  to false, incurring a cost of 45.

**Theorem 1.** Algorithm 1 is correct.

**PROOF.** Algorithm 1 recursively calls JoinAssumptions on each library in the dependency tree on line 11 to construct the formula  $F(a_1, \dots, a_n)$  that represents the assumptions being made, then uses the above encoding to create the constraint  $\text{valid}(a_1, \dots, a_n)$ . The algorithm encodes that the cost of assumptions is minimized subject to the constraint  $\text{valid}(a_1, \dots, a_n)$  on line 18.  $\square$

Although it is correct, the use of quantifiers makes the first algorithm computationally expensive; it generates a linear number of nested quantifiers in the size of the dependency tree, which have to be expanded before applying a SAT solver. While some of this complexity is unavoidable (see Section 3.4), to address this we next propose an algorithm that does not use quantifiers, and which we have found to be more effective in practice.

### 3.3 Horn clause algorithm

The key idea of Algorithm 2 is to restrict the set of assumptions to be acyclic definite Horn clauses; in this case, the problem can be solved efficiently with a binary search reduction using quantifier-free queries to an SMT solver. The set of assumptions that can be expressed as acyclic definite Horn clauses is sufficient for all of the assumptions we identify based on prior work (Table 2a).

In the running example, observe that the set of conjuncts of  $F$  that establish the safety of the `aho_corasick` library ( $c$ ) are  $a_1 \implies c$ ,  $a_4 \implies (d \implies c)$ , and  $a_7 \implies ((a \wedge m) \implies c)$ . Rewritten as Horn clauses with a body and a head, these conjuncts are  $a_1 \implies c$ ,  $(a_4 \wedge d) \implies c$ , and  $(a_7 \wedge a \wedge m) \implies c$ . Therefore, with the goal of showing that  $c$  is true, the formula  $a_1 \vee (a_4 \wedge d) \vee (a_7 \wedge a \wedge m)$  must evaluate to true. The only conjunct of  $F$  that establishes  $d$  is  $a_3 \implies d$ . Therefore, we can substitute  $d$  with  $a_3$  in the formula, resulting in  $a_1 \vee (a_4 \wedge a_3) \vee (a_7 \wedge a \wedge m)$ . By repeating this unfolding process, we can simplify the formula to  $a_1 \vee (a_4 \wedge a_3) \vee (a_7 \wedge a_5 \wedge a_6 \wedge a_8)$ . This formula will evaluate to true if and only if the assumptions made entail the conclusion  $c$ . Algorithm 2 uses this observation to solve the minimum trust problem when the assumptions are acyclic definite Horn clauses. Due to binary search, Algorithm 2 produces a logarithmic number of calls to the SMT solver with respect to the maximum cost, and each call is quantifier-free.

**Theorem 2.** Algorithm 2 is correct for assumptions consisting of acyclic definite Horn clauses.

**PROOF.** Algorithm 2 first constructs  $E$  as the disjunction of all the bodies of the Horn clauses whose heads are the conclusion  $c$  (line 10). The conclusion is provable from the assumptions if and only if  $E$  evaluates to true. Algorithm 2 then proceeds to iterate through  $V$  with the goal of eliminating all the propositional variables in  $V$  from  $E$ . For each propositional variable  $v_i$  in  $V$ , the algorithm constructs a formula  $T$  consisting of the disjunction of the bodies of the Horn clauses whose heads are  $v_i$  and then eliminates  $v_i$  by substituting  $T$  in place of  $v_i$  in  $E$ , similar to the construction of  $E$  on line 10. After the substitution,  $E$  will still be a necessary and sufficient condition for the conclusion to be true. The substitution is then applied to each remaining clause in line 19; since the Horn clauses are acyclic, eliminating  $v_i$  is guaranteed to remove  $v_i$  from all future substitutions. This process is repeated until all propositional variables in  $V$  are eliminated from  $E$ . Each step in `Unfold` maintains that  $E$  is necessary and sufficient for the conclusion to be true. The formula generated by `Unfold` is used in a binary search to repeatedly query  $S$  for valid minimum trust solutions with different costs until the minimum cost is found.  $\square$

It should be noted that the unfolding procedure in Algorithm 2 is similar to the well-known SLD resolution algorithm for logic programming [31]. Unfortunately, the generated formula has an exponential number of clauses in the worst case (with respect to the number of crates in the dependency tree). This formula, however, is in disjunctive normal form (DNF) after minor simplification, which means the SMT solver can solve each clause independently. Note that it is also possible to have a reduction that produces a polynomial-size formula, though this is not the reduction we use in our implementation.

### 3.4 Complexity

To establish the complexity of the minimum trust problem (Definition 1), we study the corresponding decision problem. To that end, define MINTRUST to be the problem that asks, given a tuple  $(\Gamma, c, C, k)$  — consisting of a set of assumptions  $\Gamma$ , a conclusion  $c$ , a cost function  $C$ , and integer  $k$  — does there exist a set of

#### Algorithm 2 Horn clause optimized minimum trust algorithm.

```

1: library: a library
2: C: a function that assigns a cost to an assumption
3: S: an SMT solver over the theory of linear integer arithmetic
4: procedure Unfold( $F$ )
5:    $F \leftarrow \text{simplify}(F)$   $\triangleright$  simplify the formula into a conjunction of
     Horn clauses
6:    $\Pi \leftarrow \text{clauses}(F)$   $\triangleright$  get the set of Horn clauses in  $F$ 
7:    $E \leftarrow \perp$   $\triangleright E$  is to be constructed such that  $E$  is true iff the
     assumptions entail the conclusion
8:   for all  $\pi_i$  in  $\Pi$  do
9:     if  $\text{head}(\pi_i)$  is the conclusion  $c$  then
10:       $E \leftarrow E \vee \text{body}(\pi_i)$   $\triangleright$  collect all the bodies of the Horn
        clauses with head  $c$ 
11:    $V \leftarrow \text{atoms}(F) \setminus \{a_1, \dots, a_n\}$   $\triangleright V$  stores the set of propositional
     variables in  $F$  other than the  $a_i$ s
12:   for all  $v_i$  in  $V$  do
13:      $T \leftarrow \perp$   $\triangleright$  the formula to be substituted in place of  $v_i$ 
14:     for all  $\pi_j$  in  $\Pi$  do
15:       if  $\text{head}(\pi_j)$  is  $v_i$  then
16:          $T \leftarrow T \vee \text{body}(\pi_j)$   $\triangleright$  collect all the bodies of the Horn
           clauses with head  $v_i$ 
17:        $E \leftarrow E[v_i/T]$   $\triangleright$  substitute  $T$  in place of  $v_i$  in  $E$ 
18:       for all  $\pi_i$  in  $\Pi$  do
19:          $\pi_i \leftarrow \pi_i[v_i/T]$   $\triangleright$  substitute  $T$  in place of  $v_i$  in  $\pi_i$ 
20:   output  $E$ 
21: procedure SolveHornMinimumTrust()
22:    $F(a_1, \dots, a_n) \leftarrow \text{JoinAssumptions}(\text{library})$   $\triangleright$  same as in
     Algorithm 1
23:    $\text{valid}(a_1, \dots, a_n) \leftarrow \text{Unfold}(F)$ 
24:    $\text{cost}(a_1, \dots, a_n) \leftarrow \sum_{i=1}^n \text{ite}(a_i, C(a_i), 0)$ 
25:    $\text{left} \leftarrow 0$   $\triangleright$  initialize the min cost as 0
26:    $\text{right} \leftarrow \sum_{i=1}^n C(a_i)$   $\triangleright$  the max cost is the sum of the costs of all
     assumptions
27:   while  $\text{left} < \text{right}$  do
28:      $k \leftarrow \lfloor (\text{left} + \text{right} + 1)/2 \rfloor$   $\triangleright$  binary search for min cost
29:      $\text{cost\_leq\_k} \leftarrow S.\text{solve}(\text{valid}(a_1, \dots, a_n) \wedge$ 
        $\text{cost}(a_1, \dots, a_n) \leq k)$ 
30:     if  $\text{cost\_leq\_k}$  is SAT then
31:        $\text{left} \leftarrow k$ 
32:     else
33:        $\text{right} \leftarrow k - 1$ 
34:    $\text{min\_cost} \leftarrow \text{left}$ 
35:   output  $S.\text{model}[a_1, \dots, a_n], \text{min\_cost}$ 

```

assumptions  $\Delta \subseteq \Gamma$  such that the sum of the costs of the assumptions in  $\Delta$  is less than or equal to  $k$  and  $\Delta \models c$ ? Similarly, define HORN-MINTRUST to be the restriction of this problem where  $\Gamma$  consists of Horn clauses. The following theorems establish that the unrestricted minimum trust problem is computationally infeasible (Theorem 3) and that the Horn-restricted minimum trust problem is intractable but still within the reach of SMT solvers (Theorem 4).

**Theorem 3.** MINTRUST is  $\Sigma_2^P$ -hard.

**PROOF.** We reduce from Shortest Implicant Core (SIC), which is known to be  $\Sigma_2^P$ -hard [54] [59]. The SIC problem asks: given a DNF formula  $\varphi$ , an implicant  $I$  of  $\varphi$  (a set of literals whose conjunction implies  $\varphi$ ), and an integer  $k$ , does an implicant  $I' \subseteq I$  of size at most  $k$  exist? Given an instance of SIC with a DNF formula  $\varphi$ , an implicant  $I$  of  $\varphi$ , and an integer  $k$ , we construct a MINTRUST

instance  $(\Gamma, c, C, k)$ . Let  $\Gamma$  contain an assumption for each literal in  $I$  with a cost of 1 and the assumption  $\varphi \implies c$  with a cost of 0.

By construction,  $\Delta \models c$  if and only if  $\Delta \models \varphi$  and  $(\varphi \implies c) \in \Delta$ . Furthermore,  $\Delta \models \varphi$  if and only if  $\Delta$  contains an implicant  $I' \subseteq I$  of  $\varphi$ . Then, the cost associated with  $\Delta$  is equal to the size of  $I'$ , so  $\varphi$  has an implicant of size at most  $k$  if and only if the MINTRUST instance has a solution of cost at most  $k$ . Therefore, an implicant  $I' \subseteq I$  of  $\varphi$  of size at most  $k$  exists if and only if this MINTRUST instance has a solution of cost at most  $k$ .  $\square$

**Theorem 4.** HORN-MINTRUST is NP-complete.

**PROOF.** *HORN-MINTRUST is NP-hard.* We reduce from Vertex Cover [28]. Given an instance of vertex cover with a graph  $G = (V, E)$  and an integer  $k$ , we construct a HORN-MINTRUST instance  $(\Gamma, c, C, k)$ . Let  $\Gamma$  contain an assumption  $v_i \implies e_j$  for each vertex  $v_i \in V$  that is incident to edge  $e_j \in E$  with cost 0, and an assumption  $(e_1 \wedge \dots \wedge e_{|E|}) \implies c$  with cost 0. By construction,  $\Delta \models c$  if and only if  $\Delta \models e_j$  for all  $e_j \in E$  and  $((e_1 \wedge \dots \wedge e_{|E|}) \implies c) \in \Delta$ . Furthermore,  $\Delta \models e_j$  for each  $e_j \in E$  if and only if  $v_i \in \Delta$  and  $(v_i \implies e_j) \in \Delta$  for some incident  $v_i$ . It follows that  $\Delta \models e_j$  for all  $e_j \in E$  if and only if  $\Delta$  is a subset of the  $v_i$ s for a vertex cover of  $G$ , together with the assumptions for incident edges. The cost associated with  $\Delta$  is equal to the size of the vertex cover, which is at most  $k$ .

*HORN-MINTRUST is in NP.* We construct a verifier for HORN-MINTRUST. On input a candidate solution  $\Delta$ , the verifier checks that the sum of the costs of the assumptions in  $\Delta$  is at most  $k$  in one pass. Since  $\Delta \models c$  holds if and only if the conjunction of the Horn clauses in  $\Delta$  with  $(c \implies \perp)$  is unsatisfiable, and Horn-satisfiability is in P, checking that  $\Delta \models c$  is possible in polynomial time.  $\square$

## 4 Design and Implementation

Cargo Sherlock is a Python-based tool for quantifying trust in the Rust supply chain ecosystem, given as input a Rust crate.<sup>3</sup> It provides a logs mode, which collects and displays information about the crate, and a trust mode (see output in Figure 1), which is used to compute the trust cost and severity label of the crate, with components as detailed in the following subsections.

### 4.1 Data collection

Cargo Sherlock first gathers crate metadata from multiple sources. From `crates.io`, it collects information about the author(s), download counts, dependencies, and GitHub repository stars and forks. From Cargo Vet [43], it sources audit reports from organizations including Google, Mozilla, and Bytecode Alliance. Cargo Sherlock also gathers the outputs from static analysis tools (Miri [53] and Cargo Scan [48, 71]) and vulnerability information from RustSec [52].

### 4.2 Relational model

The aggregated data is organized into a relational model as shown in Figure 3. The primary *entities* include: Crates, Authors, Audits, Dependencies, and Tool Results. Each Crate includes basic metadata such as its name, version, downloads, and related git data (e.g.,

stars and forks). Crates can have multiple associated tool results, audits, authors, and dependencies, following a one-to-many pattern. Dependencies connect crates to other crates, while Audits link organizations and trust criteria to crates.

### 4.3 Selection of assumptions

Our candidate assumptions are motivated by prior work on software supply chain trust. Hamer et al. [21] identified key trust factors among Rust developers, while Boughton et al. [6] decomposed these into distinct dimensions. Table 2 shows some of the assumptions we support and their justifications in prior research.

Cargo Sherlock supports three kinds of assumptions: positive, negative, and parameterized. The costs assigned to each candidate assumption are configurable by the user. In our implementation, we provide default costs, which we arrived at through a process of trial and error.

*Positive assumptions:* These are candidate assumptions a user might use to reason that a crate is safe. By default, Cargo Sherlock includes the following:

- **Trusted Author:** A crate is safe if it is authored by a well-established developer in the Rust community [21]. Cargo Sherlock includes a default list of trusted developers, which users can customize.
- **Audit Passed:** A crate is safe if it has passed an audit [21]. This assumption has a variant for crates with a previous version that have passed an audit.
- **Side Effects and Safe Dependencies:** A crate is safe if it has no *side effects*, i.e., no observable change to program state or the external environment beyond returning a value, and all of its dependencies are safe [56, 71]. Side effects are identified using Cargo Scan [48].
- **Crate is Safe:** By default, a crate is safe (this assumption has the highest possible cost). Cargo Sherlock includes this assumption with no justification as a base case.

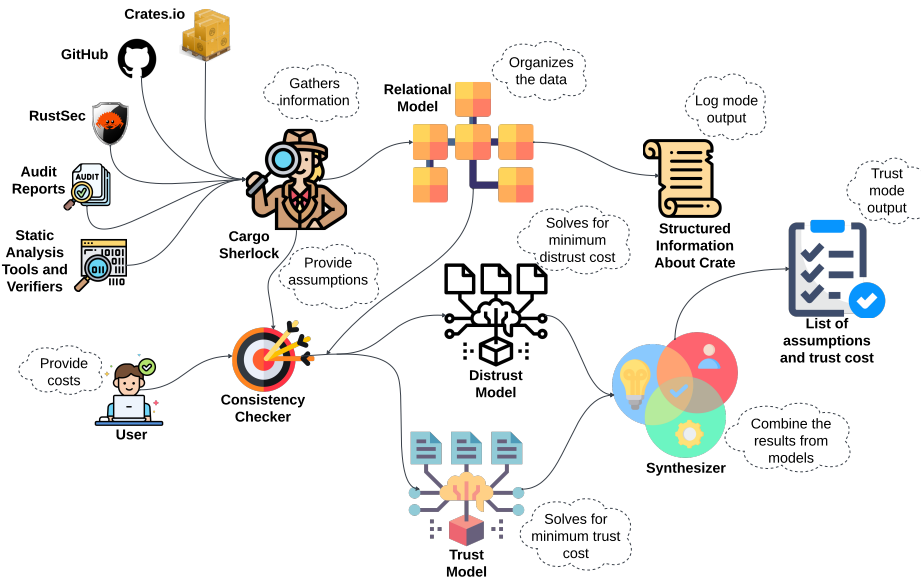
*Negative assumptions:* These are candidate assumptions a user might use to reason that a crate is unsafe. By default, Cargo Sherlock includes the following:

- **RustSec:** A crate is unsafe if it appears in the RustSec advisory database [52]. This assumption has variants for crates with a previous version that appear in RustSec and for different RustSec labels.
- **Tool Results:** A crate is unsafe if it is flagged by a static analysis tool [6]. Currently, Cargo Sherlock supports the assumption that crates flagged by Miri [53] are unsafe.
- **Unsafe Dependency:** A crate is unsafe if it has an unsafe dependency. [6]
- **Crate is Unsafe:** By default, a crate is unsafe (this assumption has the highest possible cost). Cargo Sherlock includes this assumption with no justification as a base case.

*Parameterized assumptions:* These are candidate assumptions for which the trust cost is calculated as a function of certain metadata parameters; we use a smoothing function so that the trust cost changes gradually with changes to the parameter. We also allow parameterized negative assumptions, which work for distrust cost in the same way. By default, Cargo Sherlock includes the following:

<sup>3</sup><https://github.com/davispl/cargo-sherlock>





**Figure 2: High level overview of Cargo Sherlock.**

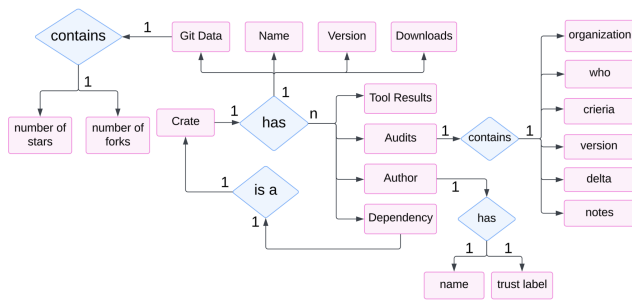
### (a) Trust model assumptions

| Cost   | Assumption                                                                  | Justification |
|--------|-----------------------------------------------------------------------------|---------------|
| 100    | The crate is safe.                                                          | Base          |
| 25-100 | If the crate has many downloads, it is safe.                                | [21] (Tbl. 7) |
| 5      | If the crate version has a passed audit, it is safe.                        | [21] (Tbl. 7) |
| 20     | If a past version has a passed audit, it is safe.                           | [21] (Tbl. 7) |
| 20-100 | If the repository has many stars/forks, it is safe.                         | [21] (Tbl. 7) |
| 10     | If the crate has no side effects and all dependencies are safe, it is safe. | [56, 71]      |
| 5      | If the author of the crate is trusted, it is safe.                          | [21] (Tbl. 7) |

### (b) Distrust model assumptions

| Cost   | Assumption                                                                                 | Justification |
|--------|--------------------------------------------------------------------------------------------|---------------|
| 100    | The crate is not safe.                                                                     | Base          |
| 5      | If the crate appears on RustSec with a critical severity label, it is not safe.            | [6]           |
| 90     | If the crate appears on RustSec and the provided crate version is patched, it is not safe. | [6]           |
| 30     | If the crate is flagged by Miri, it is not safe.                                           | [6]           |
| 60–100 | If the crate has many side effects, it is not safe.                                        | [56, 71]      |
| 10     | If the crate has an unsafe dependency, it is not safe.                                     | [6]           |

**Table 2: Selected assumptions used by Cargo Sherlock (including justifications in prior work).**



**Figure 3: Cargo Sherlock entity-relationship (ER) model.**

- **Number of Downloads:** A crate with many downloads is safe [21].
- **GitHub Stars and Forks:** A crate with many stars and forks on its GitHub is safe [21].
- **Side Effects:** A crate with many side effects is unsafe [56, 71].

Cargo Sherlock applies a *consistency checker* to ensure checks on all assumption costs; currently, it checks that no assumption has a cost higher than the base cost of 100.

#### 4.4 Computing the trust cost

We use the Z3 SMT solver [9] to compute minimum trust and distrust costs using either Algorithm 1 or Algorithm 2 (default). In our implementation, we replace the explicit binary search over costs with Z3’s optimizing SAT interface to obtain the minimum cost directly. The trust cost ranges from 0 to 100, where a lower trust cost represents a crate that is easier to trust. Similarly, the distrust cost ranges from 0 to 100, where a lower distrust cost represents a crate that is easier to distrust. The final label is derived by combining both costs, using the function illustrated in Figure 4.

## 5 Evaluation

In this section, we evaluate Cargo Sherlock on the following research questions:

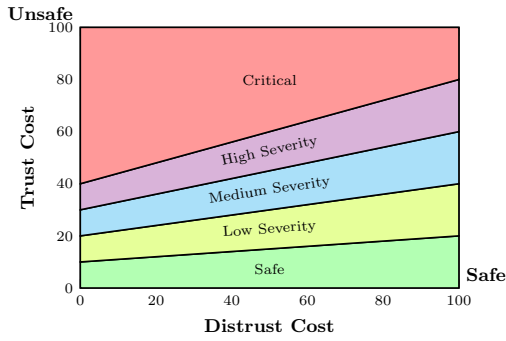


Figure 4: Cargo Sherlock combines trust and distrust costs to produce a severity label.

- **RQ<sub>1</sub>:** Does Cargo Sherlock detect synthetically generated typosquatting attacks?
- **RQ<sub>2</sub>:** Does Cargo Sherlock detect real-world supply chain risks?
- **RQ<sub>3</sub>:** How does Cargo Sherlock output fare on known vulnerable crates?
- **RQ<sub>4</sub>:** How does the performance of Algorithms 1 and 2 scale with the number of dependencies?

*Experimental setup.* All evaluations were conducted on an M2-Pro MacBook Pro with 16 GB of RAM. Our experiments are run on various sets of crates as described below: the 100 most downloaded crates from crates.io, a random sample of 1000 crates, a sample of crates flagged in the RustSec database, and two case studies. All experiments were run between March and November 2025.

*User model.* For all experiments, we adopt an idealized “rational developer” model in the sense of Dimoulas et al. [13]. The rational developer interacts with our tool as follows: when they intend to add a dependency to their existing Rust project, they execute Cargo Sherlock on the selected crate. This generates a report accompanied by a severity label; we assume that the developer looks at the report and decides whether to go forward with adding the dependency based on the severity label. For each experimental section, we interpret our findings through this user model and discuss the resulting implications for prospective users of Cargo Sherlock.

### 5.1 RQ<sub>1</sub>: Does Cargo Sherlock detect synthetically generated typosquatting attacks?

*Typosquatting* is an attack technique that exploits typographical errors to deceive users into downloading a malicious package, such as downloading `serde_yaml` in place of `serde_yaml`. We consider typosquatting because it is a common category of supply chain attack, and aligns closely with Cargo Sherlock’s intended use case. To address RQ<sub>1</sub>, we constructed synthetic typosquatted versions of the top 100 most frequently downloaded Rust crates from crates.io. We selected popular crates as a test sample because their popularity makes them a lucrative target for an attacker. We generated typosquatted variants by modifying the name and author of each crate, while keeping the functionality unchanged. To determine

| Typosquatted Crate Severity Label | CRITICAL        | 45                        | 27 |
|-----------------------------------|-----------------|---------------------------|----|
|                                   | HIGH_SEVERITY   | 3                         | 0  |
|                                   | MEDIUM_SEVERITY | 0                         | 0  |
|                                   | LOW_SEVERITY    | 0                         | 0  |
|                                   | SAFE            | 25                        | 0  |
|                                   |                 | SAFE LOW_SEVERITY         |    |
|                                   |                 | Real Crate Severity Label |    |

(a) Cargo Sherlock results on synthetically generated typosquatting attack crates (compared to their real versions).

| Crate                                                                   | Trust Cost | Distrust Cost | Severity Label  |
|-------------------------------------------------------------------------|------------|---------------|-----------------|
| <b>Pair 1: fast_log (original) vs faster_log (typosquatted)</b>         |            |               |                 |
| fast_log-1.7.7                                                          | 37         | 73            | MEDIUM_SEVERITY |
| faster_log-1.7.8                                                        | 75         | 80            | CRITICAL        |
| <b>Pair 2: serde_yaml (original) vs serde_yaml (AI-maintained fork)</b> |            |               |                 |
| serde_yaml-0.9.33                                                       | 6          | 70            | SAFE            |
| serde_yaml-0.0.12                                                       | 32         | 70            | LOW_SEVERITY    |

(b) Cargo Sherlock results on real-world incidents.

Figure 5: Sections 5.1 and 5.2 experimental results.

how many downloads would be obtained by a lesser-known crate by default, we measured that placeholder crates on crates.io received an average of 171 downloads within 3 months. We use this number for the download count in our synthetic experiment. After creating typosquatted variants, we evaluated them using Cargo Sherlock and compared their severity labels with corresponding original crates. Figure 5a shows the distribution of obtained severity labels. Most of the original crates are classified as safe (73%) and no crate is classified higher than low\_severity. On the other hand, among typosquatted crates, most are classified as critical. However, 25 typosquatted crates also receive the safe label, likely because they have no side effects and they all have safe dependencies. The severity label increases for the typosquatted crate when compared to the original crate in all other cases.

**RQ<sub>1</sub> Findings:** Most typosquatted versions of popular Rust crates incur substantially higher trust costs and higher severity labels compared to their original counterparts when evaluated by Cargo Sherlock. **Implication:** A user interacting with the tool would identify the typosquatted crate variant when adding a dependency to their code.

### 5.2 RQ<sub>2</sub>: Does Cargo Sherlock detect real-world supply chain risks?

We address this question as a case study on Rust crates that have been involved in known incidents on crates.io. We obtained



the source code for two such crates that exist to our knowledge: `faster_log` and `serde_yaml`.<sup>4</sup>

We first examine `faster_log`, an identified malicious typosquat of `fast_log` [47]. Beyond reproducing the original crate’s logging functionality, it injects code that scans log files for sensitive material (including cryptocurrency private keys) and exfiltrates any matches to a remote endpoint. Cargo Sherlock assigns a `MEDIUM_SEVERITY` label (trust cost 37, distrust cost 73) to `fast_log`, basing the trust on the author (they have a history of publishing crates) and the distrust on the presence of side effects (it performs file I/O). In contrast, Cargo Sherlock assigns `faster_log` a `CRITICAL` label, with a substantially higher trust cost (75) and elevated distrust cost (80). The change in authorship (the author had no other crates) and side effects leads to a different trust and distrust cost.

Our second case is `serde_yaml`, a fork of `serde_yaml` (the latter authored by David Tolnay, a well-established and trusted developer in the Rust community). After the original crate was marked unmaintained, the fork was published on `crates.io` and had accumulated over 1,057,000 downloads by March 19, 2025 [57]. The fork incorporates a substantial amount of AI-generated code, including low-quality “AI slop” changes that were later criticized by Tolnay [57]. As shown in Table 5b, the original crate receives a `SAFE` label, while the fork is classified with the higher `LOW_SEVERITY`, primarily due to the change in authorship. The distrust costs are similar across the two crates since no identifiable malicious code or side effects were introduced.

**RQ<sub>2</sub> Findings:** Cargo Sherlock assigns higher severity labels to the malicious typosquat `faster_log` and AI-maintained fork `serde_yaml`, relative to their original crates. **Implication:** Users interacting with Cargo Sherlock may perceive these crates as risky.

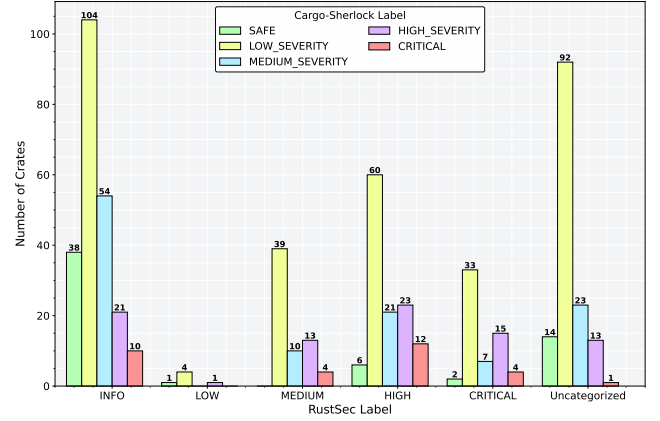
### 5.3 RQ<sub>3</sub>: How does Cargo Sherlock output fare on known vulnerable crates?

To address RQ<sub>3</sub>, we compare Cargo Sherlock’s severity labels with those from RustSec [52], a well-known database of security advisories for Rust crates. RustSec classifies crates by severity levels: critical, high, medium, low, and informational. We analyzed 592 unique crate-version pairs listed in RustSec.<sup>5</sup> For this experiment, all negative assumptions related to RustSec were removed to determine what labels would have been assigned to vulnerable crates prior to them being known to be vulnerable.

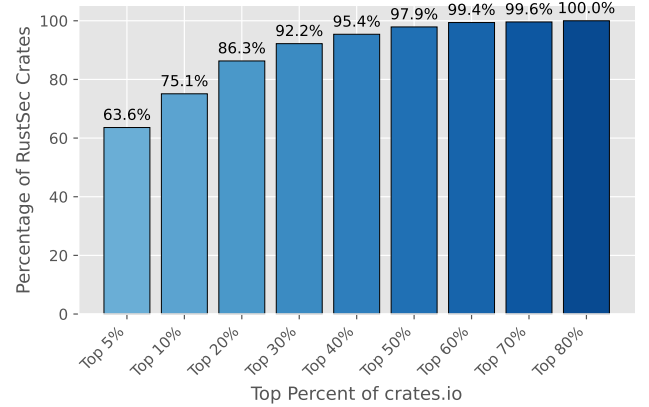
Figure 6a illustrates the distribution of severity labels for crates listed on RustSec. The x-axis shows the severity labels assigned by RustSec, and each bar is split according to the labels assigned by Cargo Sherlock. While some crates receive a higher severity label, most crates are classified as `SAFE` or `LOW_SEVERITY`. For example, these include popular crates (hence, more likely to be marked `SAFE`) that are listed in RustSec with an `INFO` label due to being marked as unmaintained. Figure 6b confirms that this behavior is generally driven by the popularity of RustSec-listed crates: approximately 75%

<sup>4</sup>We are aware of known incidents for at least two other Rust crates: `rustdecimal` [51] and `asynprintln` [47]. Unfortunately, we were unable to obtain the source code for the typosquatted versions of these crates to include them in the analysis.

<sup>5</sup>This number is as of March 25, 2025.



(a) Distribution of Cargo Sherlock vs. RustSec severity labels.



(b) Distribution of crates in RustSec by downloads.

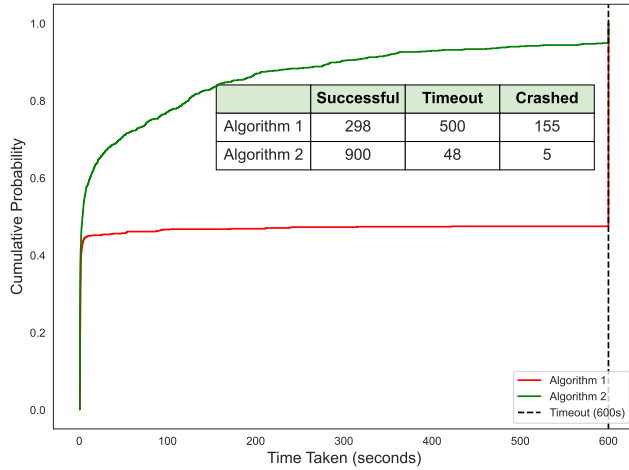
Figure 6: Section 5.3 experimental results.

of RustSec crates belong to the top 10% of `crates.io` by downloads, and only around 15% of RustSec crates are found in the bottom 80%.

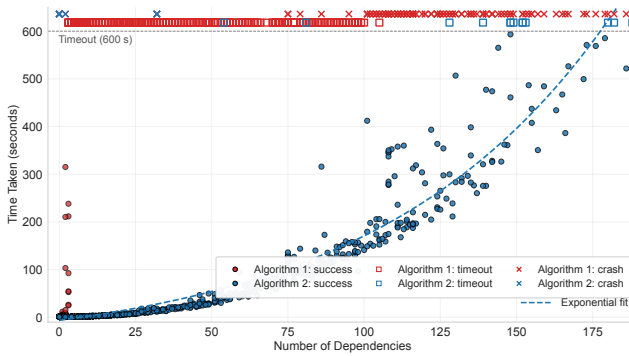
**RQ<sub>3</sub> Findings:** Crates that are marked `SAFE` by Cargo Sherlock are not necessarily free from vulnerabilities, especially in cases with a high number of downloads. **Implication:** Users interacting with Cargo Sherlock might perceive some vulnerable crates as safe.

### 5.4 RQ<sub>4</sub>: How does the performance of Algorithms 1 and 2 scale with the number of dependencies?

To address RQ<sub>4</sub>, we sampled 1,000 random crates from `crates.io`. We excluded 47 crates from our evaluation because all of their versions had been yanked, leaving 953 crates. To isolate algorithmic running time, we excluded data collection time from all measurements. The cumulative distribution function (CDF) plot in Figure 7a shows the runtime for both algorithms, using a timeout of 600 seconds. Algorithm 1 successfully evaluates 298 crates and timed out on 500 crates. It crashes on 155 crates, out of which 150 crashes are



(a) CDF of running time for Algorithm 1 vs. Algorithm 2.



(b) Time taken vs. number of dependencies.

Figure 7: Section 5.4 scalability results.

due to the system’s maximum recursion depth being exceeded. The remaining 5 crashes are shared by both algorithms and are due to parsing failures. Algorithm 2 successfully evaluated 900 crates and only timed out on 48 crates.

The scatter plot (Figure 7b) shows that Algorithm 1 rapidly becomes infeasible as the number of dependencies increases, with most evaluations timing out beyond approximately 10 dependencies. Conversely, Algorithm 2 scales better, successfully evaluating crates with up to 150 dependencies. A regression fit suggests that the scaling of the Horn clause algorithm is exponential. This is expected, as unfold in Algorithm 2 generates an exponential DNF formula in the worst case.

**RQ<sub>4</sub> Findings:** Algorithm 1 becomes infeasible after 10 dependencies, while Algorithm 2 scales to at least 150 dependencies.  
**Implication:** Users can run Cargo Sherlock on crates with a large number of dependencies and receive results within 10 minutes.

## 6 Related Work

**Supply chain security:** Software supply chain attacks [45, 46, 64] such as the SolarWinds attack [40] and XZ Utils backdoor [14, 72] have received attention from researchers and industry. Tools like Crev [12] and JFrog [7] provide support for auditing dependencies and tracking associated metadata. Software bill of materials (SBOM) [23, 44, 66, 67] is a general set of techniques for providing an overview of software contents, which can be used for human understanding; these are similar to the relational model used in our work. Tools have also been built for specific languages such as JavaScript [8, 68, 70] and Go [16, 17]. Especially relevant to our work, a few techniques in this space have used formal methods: Koscinski and Mirakhorli [30] propose a technique to formally model Common Weakness Enumerations (CWEs) using an encoding in the Alloy specification language. Software configuration management (SCM) [69] is also related; it aims to identify a configuration of dependencies that satisfies formal constraints. Our work can be thought of as a similar configuration problem to optimize for trust cost, and selecting among multiple possible configurations (different versions of different software packages) would be an interesting direction for future work.

**Safety and security for Rust code:** RustSec [52] and Lib.rs [36] provide helpful databases of known vulnerabilities and libraries. Cargo Vet [43], Cargo Audit [50], Cargo Crev [11], and Cargo Scan [48, 71] (which inspired the side effects metrics used in our tool) are tools to assist with human auditing for Rust code. Formal verification for Rust includes work on foundations — like RustBelt [27], Oxide [65], BoCa [63], Stacked/Tree Borrows [26, 62], and others [18] — and tools such as Verus [34], Prusti [2], Kani [61], and Flux [35]. Program analysis tools like MIRChecker [38], Miri [53], and Rudra [3] detect memory safety and undefined behaviors in Rust code, and fuzzing tools such as SyRust [55] and RULF [25] use automated testing to identify bugs. Sandboxing tools (e.g., [1, 4, 20, 29, 33, 39, 49]) typically aim to protect Rust code from its *unsafe* subset. However, even the *safe* subset of Rust can be exploited, and this is sometimes missed by prior tools [22, 60, 71], which motivates work incorporating human factors.

**Trust in software engineering:** Hamer et al. [21] explored the impact of contributor reputation and other indicator metrics in ecosystems like Rust and GitHub. Boughton et al. [6] proposed a framework to decompose and quantify trust within open-source software. We used these studies to motivate the specific assumptions that we support in our minimum trust cost model. Devanbu et al. [10] argue for a combination of trusted hardware, key management, and verification techniques to elevate trust. Toone et al. [58] introduce trust mediation, a framework for managing complex distributed trust relationships. Outside of computer science, models of trust have been studied in other settings (e.g., [41]).

## Acknowledgments

This research was supported in part by the US National Science Foundation under award CCF #2327338. The authors would like to thank the anonymous reviewers and Prem Devanbu for feedback leading to improvements to the paper, and Audrey Gobaco for helpful discussions and work on related projects.

## References

- [1] Hussain M. J. Almohri and David Evans. 2018. Fidelius Charm: Isolating Unsafe Rust Code. In *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy* (Tempe, AZ, USA) (CODASPY '18). Association for Computing Machinery, New York, NY, USA, 248–255. doi:10.1145/3176258.3176330
- [2] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J Summers. 2019. Leveraging Rust types for modular specification and verification. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–30.
- [3] Yechan Bae, Youngsuk Kim, Ammar Askar, Jungwon Lim, and Taesoo Kim. 2021. Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) (SOSP '21). Association for Computing Machinery, New York, NY, USA, 84–99. doi:10.1145/3477132.3483570
- [4] Inyoung Bang, Martin Kayondo, HyunGon Moon, and Yunheung Paek. 2023. TRust: A Compilation Framework for In-process Isolation to Protect Safe Rust against Untrusted Code. In *32nd USENIX Security Symposium* (USENIX Security 23). USENIX Association, Anaheim, CA, 6947–6964. <https://www.usenix.org/conference/usenixsecurity23/presentation/bang>
- [5] Stefamno Bistarelli, Fabio Martinelli, and Francesco Santini. 2008. Weighted Datalog and Levels of Trust. In *Proceedings of the 2008 Third International Conference on Availability, Reliability and Security* (ARES '08). IEEE Computer Society, USA, 1128–1134. doi:10.1109/ARES.2008.197
- [6] Lina Boughton, Courtney Miller, Yasemin Acar, Dominik Wermke, and Christian Kästner. 2024. Decomposing and Measuring Trust in Open-Source Software Supply Chains. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, Portugal) (ICSE-NIER '24). Association for Computing Machinery, New York, NY, USA, 57–61. doi:10.1145/3639476.3639775
- [7] Stephen Chin. 2023. Closing the Supply Chain Security Loop with Rust @ Rust Nation UK | JFrog. <https://jfrog.com/community/rust/closing-the-supply-chain-security-loop-with-rust-and-pyrsia/>. [Accessed 2023-12].
- [8] Filipe Roseiro Cogo, Gustavo A Oliva, and Ahmed E Hassan. 2019. An empirical study of dependency downgrades in the npm ecosystem. *IEEE Transactions on Software Engineering* 47, 11 (2019), 2457–2470.
- [9] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340.
- [10] Premkumar T. Devanbu, Philip W-L Fong, and Stuart G. Stubblebine. 1998. Techniques for trusted software engineering. In *Proceedings of the 20th International Conference on Software Engineering* (Kyoto, Japan) (ICSE '98). IEEE Computer Society, USA, 126–135.
- [11] Crev development team. 2024. GitHub – crev-dev/cargo-crev: A cryptographically verifiable code review system for the cargo (Rust) package manager. <https://github.com/crev-dev/cargo-crev>. [Accessed 14-11-2024].
- [12] Crev development team. 2024. GitHub – crev-dev/crev: Crev - Code REView system that we desperately need. <https://github.com/crev-dev/crev>. [Accessed 14-11-2024].
- [13] Christos Dimoulas and Matthias Felleisen. 2025. The Rational Programmer: Investigating Programming Language Pragmatics. *Commun. ACM* 68, 7 (2025), 120–130.
- [14] Andres Freund. 2024. (oss-security) backdoor in upstream xz/liblzma leading to ssh server compromise. <https://www.openwall.com/lists/oss-security/2024/03/29/4>. [Accessed 05-11-2025].
- [15] Seyed Mohammad Ghaffarian and Hamid Reza Shahriari. 2017. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey. *ACM computing surveys (CSUR)* 50, 4 (2017), 1–36.
- [16] Google. 2023. Capslock: What is your code really capable of? <https://security.googleblog.com/2023/09/capslock-what-is-your-code-really.html>. [Accessed 14-11-2024].
- [17] Google. 2023. GitHub – Google Capslock. <https://github.com/google/capslock>. [Accessed 14-11-2024].
- [18] Zachary Grannan, Aurel Bilý, Jonáš Fiala, Jasper Geer, Markus de Medeiros, Peter Müller, and Alexander J Summers. 2025. Place Capability Graphs: A General-Purpose Model of Rust's Ownership and Borrowing Guarantees. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 2002–2029.
- [19] Todd J. Green and Val Tannen. 2017. The Semiring Framework for Database Provenance. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (Chicago, Illinois, USA) (PODS '17). Association for Computing Machinery, New York, NY, USA, 93–99. doi:10.1145/3034786.3056125
- [20] Merve Gulmez, Thomas Nyman, Christoph Baumann, and Jan Tobias Muhlberg. 2023. Friend or Foe Inside? Exploring In-Process Isolation to Maintain Memory Safety for Unsafe Rust. In *2023 IEEE Secure Development Conference (SecDev)*. IEEE Computer Society, Los Alamitos, CA, USA, 54–66. doi:10.1109/SecDev56634.2023.00020
- [21] Sivana Hamer, Nasif Imtiaz, Mahzabin Tamanna, Preya Shabrina, and Laurie Williams. 2025. Trusting Code in the Wild: Exploring Contributor Reputation Measures to Review Dependencies in the Rust Ecosystem. *IEEE Trans. Softw. Eng.* 51, 4 (April 2025), 1319–1333. doi:10.1109/TSE.2025.3551664
- [22] Muhammad Hassnain and Caleb Stanford. 2024. Counterexamples in Safe Rust. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops* (Sacramento, CA, USA) (ASEW '24). Association for Computing Machinery, New York, NY, USA, 128–135. doi:10.1145/3691621.3694943
- [23] Stephen Hendrick and J Zemlin. 2022. *The state of software bill of materials (SBOM) and cybersecurity readiness*. Technical Report. Retrieved 2023-08-18 from <https://www.linuxfoundation.org/research/the-...>
- [24] Federico Heras, Javier Larrosa, and Albert Oliveras. 2008. MiniMaxSAT: An efficient weighted Max-SAT solver. *Journal of Artificial Intelligence Research* 31 (2008), 1–32.
- [25] Jianfeng Jiang, Hui Xu, and Yangfan Zhou. 2022. RULF: Rust library fuzzing via API dependency graph traversal. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering* (Melbourne, Australia) (ASE '21). IEEE Press, Los Alamitos, CA, USA, 581–592. doi:10.1109/ASE51524.2021.9678813
- [26] Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. 2019. Stacked Borrows: An Aliasing Model for Rust. *Proc. ACM Program. Lang.* 4, POPL, Article 41 (dec 2019), 32 pages. doi:10.1145/3371109
- [27] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2017. RustBelt: Securing the Foundations of the Rust Programming Language. *Proc. ACM Program. Lang.* 2, POPL, Article 66 (dec 2017), 34 pages. doi:10.1145/3158154
- [28] Richard Karp. 1972. Reducibility Among Combinatorial Problems. *Complexity of Computer Computations* 40 (01 1972), 85–103. doi:10.1007/978-3-540-68279-0\_8
- [29] Paul Kirth, Mitchel Dickerson, Stephen Crane, Per Larsen, Adrian Dabrowski, David Gens, Yeoul Na, Stijn Volckaert, and Michael Franz. 2022. PKRU-Safe: Automatically Locking down the Heap between Safe and Unsafe Languages. In *Proceedings of the Seventeenth European Conference on Computer Systems* (Rennes, France) (EuroSys '22). Association for Computing Machinery, New York, NY, USA, 132–148. doi:10.1145/3492321.3519582
- [30] Viktoria Koscinski and Mehdi Mirakhorli. 2024. Formally Modeled Common Weakness Enumerations (CWEs). In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops* (Sacramento, CA, USA) (ASEW '24). Association for Computing Machinery, New York, NY, USA, 88–93. doi:10.1145/3691621.3694938
- [31] Robert A Kowalski. 1974. Predicate Logic as a Programming Language. *IFIP Congress* 74 (1974), 569–574.
- [32] Ivan Victor Krsul. 1998. *Software vulnerability analysis*. Purdue University, West Lafayette, IN, USA.
- [33] Benjamin Lamowski, Carsten Weinhold, Adam Lackorzynski, and Hermann Härtig. 2017. Sanderust: Automatic Sandboxing of Unsafe Components in Rust. In *Proceedings of the 9th Workshop on Programming Languages and Operating Systems* (Shanghai, China) (PLOS '17). Association for Computing Machinery, New York, NY, USA, 51–57. doi:10.1145/3144555.3144562
- [34] Verus language team. 2022. verus-lang/verus: Verified Rust for low-level systems code. <https://github.com/verus-lang/verus>. [Accessed 2023-2-03].
- [35] Nico Lehmann, Adam T Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid types for Rust. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1533–1557.
- [36] Kornel Lesiński. 2024. Lib.rs – home for Rust crates. <https://lib.rs/>. [Accessed 14-11-2024].
- [37] Chu Min Li and Felip Manyà. 2009. MaxSAT, hard and soft constraints. In *Handbook of satisfiability*. IOS Press, Amsterdam, The Netherlands, 613–631.
- [38] Zhuohua Li, Jincheng Wang, Mingshen Sun, and John C.S. Lui. 2021. MirChecker: Detecting Bugs in Rust Programs via Static Analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) (CCS '21). Association for Computing Machinery, New York, NY, USA, 2183–2196. doi:10.1145/3460120.3484541
- [39] Peiming Liu, Gang Zhao, and Jeff Huang. 2020. Securing Unsafe Rust Programs with XRust. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (Seoul, South Korea) (ICSE '20). Association for Computing Machinery, New York, NY, USA, 234–245. doi:10.1145/3377811.3380325
- [40] Jefferson Martínez and Javier M Durán. 2021. Software supply chain attacks, a threat to global cybersecurity: SolarWinds' case study. *International Journal of Safety and Security Engineering* 11, 5 (2021), 537–545.
- [41] Roger C Mayer, James H Davis, and F David Schoorman. 1995. An integrative model of organizational trust. *Academy of management review* 20, 3 (1995), 709–734.
- [42] Anders Möller and Michael I Schwartzbach. 2012. Static program analysis. Lecture notes. February 2012.
- [43] Mozilla. 2024. cargo-vet: Supply-Chain Security for Rust. <https://github.com/mozilla/cargo-vet>. [Accessed 05-11-2024].
- [44] Éamonn Ó Muirí. 2019. Framing software component transparency: Establishing a common software bill of material (SBOM). National Telecommunications and Information Administration. November 2019.
- [45] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks.

- In *Detection of Intrusions and Malware, and Vulnerability Assessment: 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings* (Lisbon, Portugal). Springer-Verlag, Berlin, Heidelberg, 23–43. doi:10.1007/978-3-030-52683-2\_2
- [46] Marc Ohm and Charlene Stuke. 2023. SoK: Practical Detection of Software Supply Chain Attacks. In *Proceedings of the 18th International Conference on Availability, Reliability and Security* (Benevento, Italy) (ARES '23). Association for Computing Machinery, New York, NY, USA, Article 33, 11 pages. doi:10.1145/3600160.3600162
- [47] Walter Pearce. 2025. crates.io: Malicious crates faster\_log and async\_println. <https://blog.rust-lang.org/2025/09/24/crates.io-malicious-crates-fasterlog-and-asyncprintln/>. Rust Blog; on behalf of the crates.io team. Accessed 2025-10-14.
- [48] UC Davis PL and UC San Diego PLSysSec. 2026. Cargo Scan: a tool for auditing Rust crates. <https://github.com/PLSysSec/cargo-scan>. [Accessed 10-09-2024].
- [49] Elijah Rivera, Samuel Mergendahl, Howard Shrobe, Hamed Okhravi, and Nathan Burow. 2021. Keeping Safe Rust Safe with Galeed. In *Proceedings of the 37th Annual Computer Security Applications Conference* (Virtual Event, USA) (ACSAC '21). Association for Computing Machinery, New York, NY, USA, 824–836. doi:10.1145/3485832.3485903
- [50] Rust Secure Code Working Group. 2024. GitHub – RustSec: cargo audit. <https://github.com/rustsec/rustsec/tree/main/cargo-audit>. [Accessed 14-11-2024].
- [51] Rust Secure Code Working Group. 2024. RUSTSEC-2022-0042: malicious crate 'rustdecimal' — RustSec Advisory Database. <https://rustsec.org/advisories/RUSTSEC-2022-0042.html>. [Accessed 03-08-2024].
- [52] Rust Secure Code Working Group. 2025. RustSec: Rust Security Advisory Database. <https://rustsec.org/advisories/>. [Accessed 06-11-2024].
- [53] Rust team. 2024. GitHub – rust-lang/miri: An interpreter for Rust's mid-level intermediate representation. <https://github.com/rust-lang/miri>. [Accessed 2023-12-06].
- [54] Marcus Schaefer and Christopher Umans. 2002. Completeness in the polynomial-time hierarchy: A compendium. *SIGACT news* 33, 3 (2002), 32–49.
- [55] Yoshiaki Takashima, Ruben Martins, Limin Jia, and Corina S. Păsăreanu. 2021. SyRust: automatic testing of Rust libraries with semantic-aware program synthesis. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) (PLDI 2021). Association for Computing Machinery, New York, NY, USA, 899–913. doi:10.1145/3453483.3454084
- [56] David Terei, Simon Marlow, Simon Peyton Jones, and David Mazières. 2012. Safe Haskell. *SIGPLAN Notices (Proceedings of the 2012 Haskell Symposium)* 47, 12 (Sept. 2012), 137–148. doi:10.1145/2430532.2364524
- [57] David Tolnay. 2025. Tweet on January 27, 2025 (topic: AI-maintained serde\_yaml). <https://x.com/davidtolnay/status/1883906113428676938>. Accessed: 2025-03-25.
- [58] Brian Toone, Michael Gertz, and Premkumar Devanbu. 2003. Trust Mediation for Distributed Information Systems. In *Security and Privacy in the Age of Uncertainty*, Dimitris Gritzalis, Sabrina De Capitani di Vimercati, Pierangela Samarati, and Sokratis Katsikas (Eds.). Springer US, Boston, MA, 1–12.
- [59] Christopher Umans. 2001. The Minimum Equivalent DNF Problem and Shortest Implicants. *J. Comput. System Sci.* 63, 4 (2001), 597–611. doi:10.1006/jcss.2001.1775
- [60] Speykious (GitHub user). 2024. cve-rs: Blazingly fast memory vulnerabilities, written in 100% safe Rust. <https://github.com/Speykious/cve-rs>. [Accessed 11-11-2024].
- [61] Alexa VanHattum, Daniel Schwartz-Narbonne, Nathan Chong, and Adrian Sampson. 2022. Verifying dynamic trait objects in Rust. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice* (Pittsburgh, Pennsylvania) (ICSE-SEIP '22). Association for Computing Machinery, New York, NY, USA, 321–330. doi:10.1145/3510457.3513031
- [62] Neven Villani, Johannes Hostert, Derek Dreyer, and Ralf Jung. 2025. Tree borrows. *Proceedings of the ACM on Programming Languages* 9, PLDI (2025), 1019–1042.
- [63] Andrew Wagner, Olek Gierczak, Brianna Marshall, John M Li, and Amal Ahmed. 2025. From Linearity to Borrowing. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 3981–4007.
- [64] Xinyuan Wang. 2021. On the Feasibility of Detecting Software Supply Chain Attacks. In *MILCOM 2021 - 2021 IEEE Military Communications Conference (MILCOM)*. IEEE, Piscataway, NJ, USA, 458–463. doi:10.1109/MILCOM52596.2021.9652901
- [65] Aaron Weiss, Olek Gierczak, Daniel Patterson, and Amal Ahmed. 2019. Oxide: The Essence of Rust. arXiv. arXiv:1903.00982
- [66] Boming Xia, Tingting Bi, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2023. An Empirical Study on Software Bill of Materials: Where We Stand and the Road Ahead. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) (ICSE '23). IEEE Press, Los Alamitos, CA, USA, 2630–2642. doi:10.1109/ICSE48619.2023.00219
- [67] Nusrat Zahan, Elizabeth Lin, Mahzabin Tamanna, William Enck, and Laurie Williams. 2023. Software bills of materials are required. Are we there yet? *IEEE Security & Privacy* 21, 2 (2023), 82–88.
- [68] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice* (Pittsburgh, Pennsylvania) (ICSE-SEIP '22). Association for Computing Machinery, New York, NY, USA, 331–340. doi:10.1145/3510457.3513044
- [69] Andreas Zeller and Gregor Snelting. 1997. Unified versioning through feature logic. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 6, 4 (1997), 398–441.
- [70] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with high risks: a study of security threats in the npm ecosystem. In *Proceedings of the 28th USENIX Conference on Security Symposium* (Santa Clara, CA, USA) (SEC'19). USENIX Association, USA, 995–1010.
- [71] Lydia Zoghbi, David Thien, Ranjit Jhala, Deian Stefan, and Caleb Stanford. 2026. Auditing Rust Crates Effectively. To appear, experience report in European Symposium on Programming (ESOP).
- [72] Zeljka Zorz. 2024. Beware! Backdoor found in XZ utilities used by many Linux distros (CVE-2024-3094) - Help Net Security. <https://www.helpnetsecurity.com/2024/03/29/cve-2024-3094-linux-backdoor/>. [Accessed 05-11-2025].